

PROJECT DOCUMENTATION

PROJECT TITLE: AI WEATHERWISE

TEAM ID: SWTID-2026-7520

TEAM LEADER:

Santhoshini S - 24cssanthoshini087@gmail.com

TEAM MEMBERS:

- 1. Sanjana B - bs4905992@gmail.com**
- 2. Saranya R- 24cssaranya89@gmail.com**
- 3. Saraswathy M-24cssaraswathym90@gmail.com**
- 4. Saral Esther Rani j - 24cssaralestherranij088@gmail.com**

1. INTRODUCTION

AI WeatherWise is a RESTful backend application developed using Node.js and Express.js. It uses MongoDB with Mongoose for data storage and integrates Google Gemini AI to provide weather summaries and personalized recommendations. The system helps users track weather information and receive useful suggestions based on current weather conditions.

The application is designed to make weather information easier to understand and more useful for users. It allows users to securely manage their accounts and favorite locations while accessing weather-related information through APIs. The integration of artificial intelligence provides meaningful summaries and recommendations, making the system more interactive and helpful.

AI WeatherWise demonstrates the use of modern backend technologies, database management, secure authentication, API development, and artificial intelligence in a single application. It also provides a foundation that can be further developed into a complete web or mobile weather application.

2. PROJECT OVERVIEW

The project provides a centralized backend for weather tracking, favorite-location management, and AI-based weather insights. Users can register and log in securely, save favorite cities, retrieve current weather information, and request AI-generated summaries and recommendations.

FEATURES

- User registration and login using JWT authentication.
- Password protection using bcrypt hashing.
- Add, view, and manage favorite locations.
- Real-time weather information retrieval.
- AI-generated weather summaries using Google Gemini.
- Activity and clothing recommendations.
- Request sanitization and centralized error handling.
- Fallback mode when required services are unavailable.

3. ARCHITECTURE

The application follows the MVC (Model-View-Controller) architecture. The API acts as the interface for client requests. Controllers handle weather retrieval and AI insights, models manage users and locations through Mongoose, and middleware provides authentication and request processing.

- API Interface – receives and responds to client requests.
- Express Gateway – manages routes and HTTP requests.
- Authentication Middleware – validates JWT tokens.
- Service Layer – handles weather and AI-related operations.
- Mongoose Database Interface – connects the application with MongoDB.
- AI Service – communicates with Google Gemini for intelligent insights.

4. SETUP INSTRUCTIONS

- Create the project folder and initialize the Node.js project using `npm init -y`.
- Install the required packages: `express`, `mongoose`, `bcryptjs`, `jsonwebtoken`, `cors`, `dotenv`, `multer`, and `@google/genai`.
- Install `nodemon` optionally for development.
- Install and configure MongoDB.
- Create a `.env` file with `PORT`, `MONGO_URI`, `JWT_SECRET`, and `GEMINI_API_KEY`.
- Open the project in Visual Studio Code.

5. FOLDER STRUCTURE

```
AI-WeatherWise/  
|  
├── config/  
|  
├── controllers/  
|  
├── middleware/  
|  
├── models/  
|  
├── routes/  
|
```

```
├── services/
|
├── utils/
|
├── .env
|
├── package.json
|
└── server.js
```

6. RUNNING THE APPLICATION

- Make sure MongoDB is running.
- Open the project folder in the terminal.
- Run npm install to install dependencies.
- Configure the .env file.
- Start the application using npm start.
- The backend runs on the configured port, such as port 5000.
- Use Postman or Thunder Client to test the API endpoints.

7. API DOCUMENTATION

- Register API – creates a new user account.
- Login API – authenticates a user and provides a JWT token.
- Weather API – retrieves current weather information.

- Recommendation API – generates AI-based weather insights and recommendations.
- Favorite Location APIs – support adding, viewing, updating, and deleting saved locations.
- The APIs communicate through HTTP requests and JSON responses. Protected endpoints require a valid JWT token.

8. AUTHENTICATION

The application uses JWT authentication. During registration, passwords are securely hashed using bcrypt. After successful login, a JWT token is generated and used to access protected API endpoints. Authentication middleware validates the token before allowing access to protected resources.

9. USER INTERFACE

AI WeatherWise is primarily a backend/API project. User interaction is handled through API requests and responses rather than a traditional graphical frontend. Postman or Thunder Client can be used to send requests and view JSON responses during testing.

10. TESTING

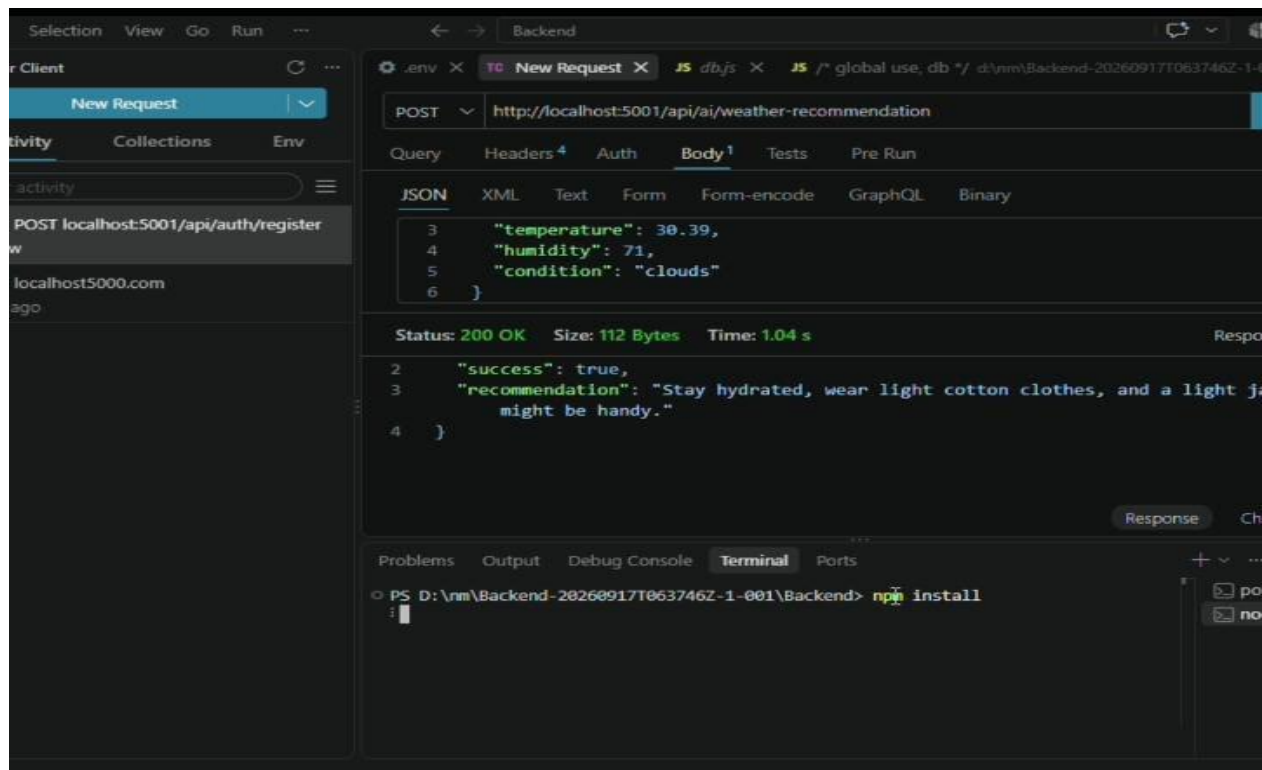
The APIs can be tested using Postman or Thunder Client. Testing includes registration, login, authentication, favorite-location operations, weather retrieval, and AI recommendation requests.

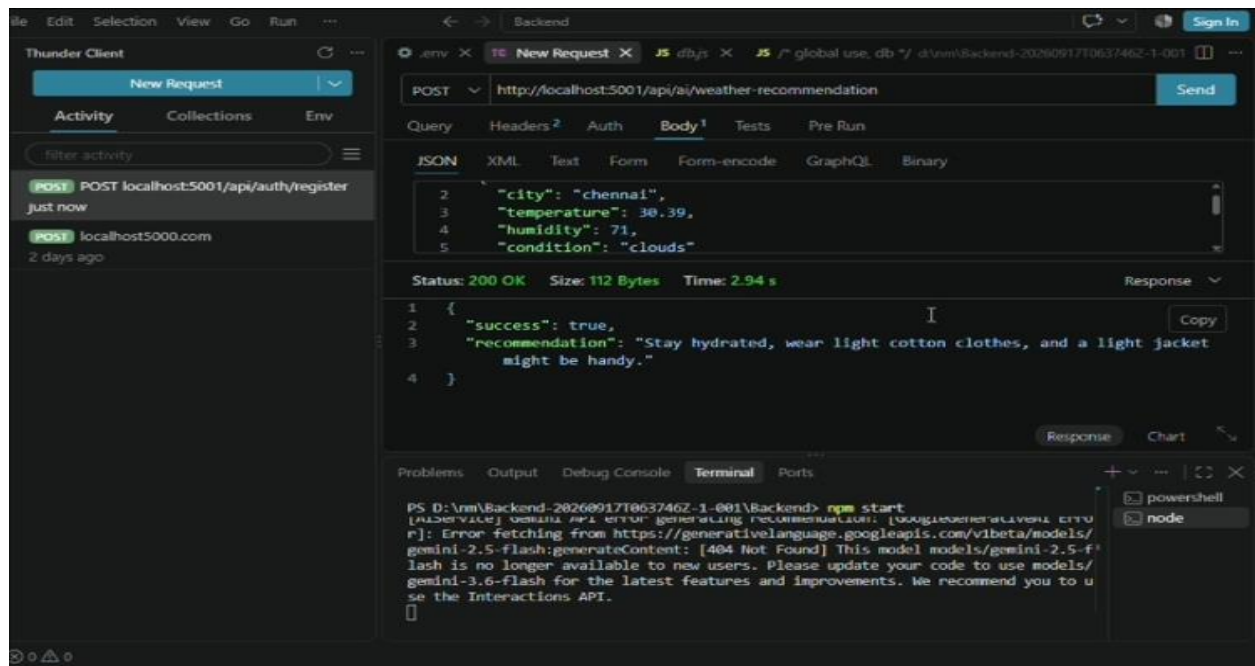
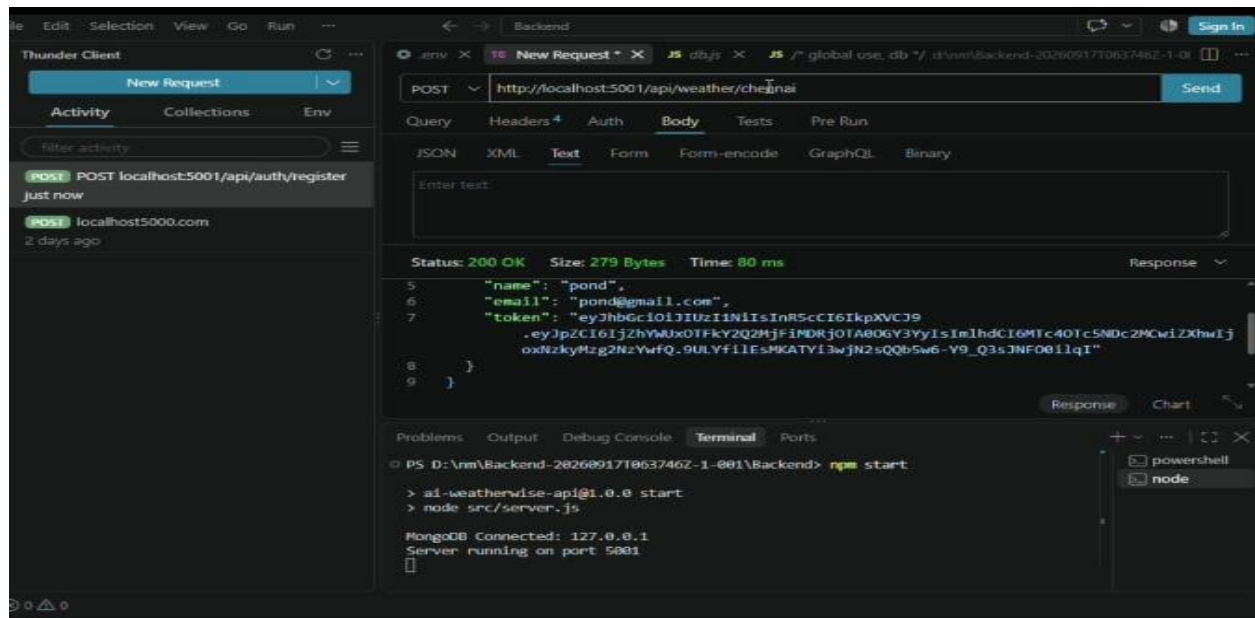
- Check valid and invalid registration data.
- Test login with correct and incorrect credentials.

- Verify JWT-protected endpoints.
- Test adding and managing favorite locations.
- Check weather API responses.
- Test AI summary and recommendation responses.
- Verify error handling and fallback behavior

11. SCREENSHOT & DEMO VIDEO

Screenshots can be added here to show API testing and application execution. Recommended screenshots include registration/login response, weather API response, recommendation response, Postman or Thunder Client testing, and backend execution in the terminal.





DEMOVIDEO LINK:

<https://drive.google.com/file/d/1mTyOV7xeBOQ4H8-BofSHwdjDdBCaAAoo/view?usp=drivesdk>

12. ISSUES

- The project depends on external weather and AI services.
- Internet connectivity is required for real-time service access.
- API keys and environment variables must be configured correctly.
- The current project is mainly backend/API based and does not include a full graphical frontend.
- Fallback handling is required when external services are unavailable.

13. FUTURE ENHANCEMENTS

- Develop a responsive web or mobile frontend.
- Add GPS-based location detection.
- Provide hourly and extended weather forecasts.
- Add weather alerts and notifications.
- Integrate maps and location visualization.
- Support multiple languages.
- Add voice-based weather interaction.

14. CONCLUSION

AI WeatherWise provides a centralized backend for weather tracking and personalized AI-based weather guidance. By combining Node.js, Express.js, MongoDB, JWT authentication, and Google Gemini AI, the project supports secure user management, favorite locations, real-time weather retrieval, and intelligent recommendations.

The system makes weather information easier to understand by providing AI-generated summaries and useful suggestions based on current weather conditions. The secure authentication system also helps protect user accounts and their saved locations. The modular architecture makes the application easier to maintain, test, and improve in the future.

Overall, AI WeatherWise demonstrates how modern backend technologies and artificial intelligence can be combined to create a useful and user-friendly weather application. The system can be further extended with a responsive web or mobile frontend, GPS-based location detection, weather alerts, maps, multilingual support, voice interaction, and extended weather forecasts.

15. REFERENCE

1. API (Application Programming Interface) – Used for communication between the application and weather-related services.
2. MongoDB – Used as the database for storing user information and favorite locations.
3. Thunder Client – Used for testing and verifying the API requests and responses.
4. Express.js & Node.js – Used to develop the backend server and create RESTful APIs.